PETAL STUDIOS · INTERNAL ENGINEERING REFERENCE

WILSON
Systems Atlas

v1.0.0 · 2026-07-30
tree 09b4405 · feat/multi-user-v1
visual counterpart to SYSTEMS_HANDBOOK.md

WHAT THIS IS

A multi-tenant creative-production platform: **one codebase, two hosts, three tools**, a company admin terminal and a separate platform-operator console. This document draws it. The handbook explains it.

HOW TO READ IT

Everything in `mono` is a real, searchable string — a table, route, function, bucket or error code. Nothing has been simplified for the diagram.

THE ONE SENTENCE

Clients are untrusted. **The database is the boundary.** Edge Functions exist only for what RLS cannot express — and every one re-checks a live row.

00a Start here — the whole thing in plain English

no jargon on this page

Think of WILSON as **one office building that several companies rent space in**.

Each company is a "workspace". Petal Studios is one; a client studio would be another. They share the same software and the same storeroom, and they must never see each other's things.

There is one storeroom, not one per company. Every box in it has the company's name written on it, and a lock that checks your badge before it opens. That lock is the thing this whole document keeps pointing at — it is called *row-level security*, and it lives in the database itself.

The apps people use are not trusted. The desktop app and the website are treated as strangers holding a badge. They can ask for anything they like; the lock decides what they actually get. That is why hiding a button is never called security here — it is only tidiness.

There are three tools inside the building — D.O.G. makes slide outlines, O.T.T.E.R. makes and teaches courses, R.A.B.B.I.T. plans productions and budgets. They share a front door, a staff list and a login, and almost nothing else.

Petal Studios also has a caretaker's office — a completely separate little website where you can create a new company or shut one down. It is not part of the normal app, and it is not inside the desktop program at all.

IF YOU REMEMBER ONE THING

The database is the security guard — **not the app**. Every screen, button and permission list you see is a convenience. If someone got past all of them, the database would still refuse to hand over another company's data.

Everywhere you see the red band in this document, that is the guard. Nothing gets from the untrusted side to the trusted side without passing through it.

HOW TO USE THIS DOCUMENT

Every section opens with a grey **"in plain terms"** strip. Read only those and you will have the whole story. Everything below each strip is the exact engineering detail — real names an engineer can search for. You do not need to read it, but it must stay exactly as written.

JARGON DECODER — EVERY TERM THIS DOCUMENT USES

left column: what engineers call it · right: what it is

Supabase
The rented back office. It provides the database, the login system, the file storage and the small server programs — all from one vendor.

Postgres
The database — the storeroom itself. Every project, task, budget line and course lives here as a row in a table.

RLS · row-level security
The lock on every individual row. The database checks, on every single request, whether *this* person is allowed *this* row. This is the security boundary.

JWT · token · claims
The badge you carry after logging in. It states who you are, which company you are in and what tier you hold. It is re-issued about once an hour — which is why anything important double-checks the real records instead of trusting the badge.

Edge Function
A small trusted program running on the back office's own machines. Used only for jobs the database cannot do itself — creating accounts, holding secrets, or acting across two companies.

service_role
The master key those small programs hold. It ignores the locks — so every one of them is written to re-check your real record before doing anything.

PostgREST
The hatch that lets the app talk to the database directly, without a middleman. Safe here *only* because the locks are on the rows.

Realtime · broadcast
The live tap on the shoulder. When a colleague edits a task, your screen updates without refreshing.

Bucket · Storage
Where actual files live — the two are `user-avatars` (profile pictures) and `rabbit-files` (project files). The database only stores a note saying where each file is.

Migration · 0000–0029
A numbered, permanent change to the storeroom's shape. They are applied in order and never edited afterwards, so any environment can be rebuilt from scratch identically.

Trigger · cron
A *trigger* is a rule that fires automatically whenever a row changes — like a logbook that writes itself. *Cron* is a nightly alarm clock that runs tidy-up jobs.

Electron
The wrapper that turns the website into an installable Windows program, and lets it reach the hard drive.

Build target · surface
One recipe for packaging the same code. The desktop app, the website and the caretaker's console are three different bakes of one dough.

Streaming · SSE
Receiving an AI answer word by word as it is written, instead of waiting for the whole thing. Used here because long answers would otherwise time out and be lost.

MFA · TOTP · aal2
The six-digit code from a phone app, on top of the password. `aal2` just means "this session did the second step".

Soft delete · purge
Deleting puts something in a bin where it can be restored. After 30 days it is destroyed for good — and a permanent receipt is written proving it happened.

pgTAP · CI
Automated tests that try to break the locks on every code change. 37 test files, 555 checks. If a lock stops working, the change is refused.

TPN · MPA
The film and TV industry's content-security standard. Studios ask whether your tools meet it before trusting you with unreleased material.

00b The visual language

used on every page after this one

CLIENT vs SERVER

Client-side.
Outlined. Untrusted.

Server-side.
Filled. Enforces.

Third party.
Named, never generic.

THE TWO HOSTS

EL — Electron desktop.
Has a local plane.

WEB — Web build.
Cloud only, degrades.

A capability struck through is ~~unavailable on that host~~ — never merely hidden.

PRIVILEGE, LOW → HIGH

- user
- manager
- admin
- **platform operator**

The fourth tier is cross-tenant, lives on a different surface, and **cannot be granted from any UI**.

INVARIANTS

FROZEN Cannot change without a coordinated migration.

CLOSED Fails closed. Refuses when it cannot verify.

OPEN Fails open, deliberately. Reason always stated.

ORDER The sequence is the design. Reordering breaks it.

01 The system map

IN PLAIN TERMS

This is the one-page picture of everything WILSON touches. The top row is the software people actually open. The red band across the middle is the security guard. Below it sits the rented back office, and below that the outside companies WILSON pays for specific jobs — sending email, storing backups, answering AI questions. **Nothing in the top row can reach the outside companies directly.** The box in red at the bottom right is just as important as the arrows: it lists the connections that deliberately do not exist.

Every external system WILSON depends on, arranged by which side of the boundary it sits on. Read it top to bottom: nothing on the client side may act on data without crossing the red band.

THE TRUST BOUNDARY

Postgres RLS

Not the client. Not a middle tier. Not the Edge Functions. Arrows into Postgres from clients are **direct** — and that is safe precisely because the boundary is here.

Auth / GoTrue

`signInWithPassword`
`mfa.challenge / verify`
`refreshSession`

ES256 JWT, minted by the access-token hook.

PostgREST + RPC

`direct table reads`
`direct table writes`
`SECURITY DEFINER RPCs`

The main data path in cloud mode. Under the caller's own token.

Realtime

`rabbit:project:{id}`
`rabbit:workspace:{id}`

Authorised **once**, at join. Broadcast, never `postgres_changes`.

Storage

`user-avatars`
`rabbit-files`

Bucket policies are RLS too. No UPDATE policy on `rabbit-files`.

Edge Functions ×12

`bearer token`
`verify_jwt = false`

The gateway only speaks HS256; these tokens are ES256. Each function verifies itself.

FROZEN `auth.jwt() → app_metadata → { workspace_id, workspace_ids, app_role, is_platform_operator }`

Every tenant-scoped policy starts here. Changing the shape means rewriting RLS on every domain table *and* re-issuing every live token.

STALENESS `jwt_expiry` is 3600 s, so claims can lag reality by up to an hour. **Reads may use the claim. Anything that acts re-checks a live row.** That is why `is_platform_operator()` reads the table and never the claim — revocation bites on the next statement, not the next refresh.

SUPABASE — SERVER SIDE

three environments: wilson-dev · wilson-staging · wilson-prod — migrations 0000–0029 on all three

Auth / GoTrue

`custom_access_token_hook`
`ES256`
`TOTP factors`

Postgres 17

`ENABLE + FORCE RLS`
`37 pgTAP suites`
`555 assertions`

Four tables are ENABLE-only, each deliberately.

Realtime

`realtime.broadcast_changes`
`10 project tables`
`5 workspace tables`

Storage

`user-avatars — public`
`rabbit-files — private`

Edge Functions

`×12`
`3 shared guards`
`1 crypto · 1 limiter`

pg_cron

`×5 nightly purges`
`04:43 → 04:59 UTC`

RLS BYPASSED `service_role`

Edge Functions run here, so **every one of them re-checks a live `workspace_members` row itself** — claims can outlive a demotion by the token TTL; the membership row cannot. Sole executor of provisioning, purges, `operator_workspace_summary()`, `fn_rate_limit_hit()` and teardown.

EXTERNAL — REACHED FROM THE SERVER SIDE, OR OUT OF BAND

← ai-proxy · operator-ai-keys

Anthropic

api.anthropic.com
stream: true, always
claude-sonnet-4-20250514
claude-haiku-4-5-20251001

← Supabase Auth (SMTP)

Resend

mail.petalstudios.co
invite · recovery · email change
→ {SITE_URL}/#/recovery

← GitHub Actions · → electron-updater

Backblaze B2

db/{env}/…dump.gz — 90d, Object Lock
updates.petalstudios.co/wilson

Never customer content. Dumps and installers only.

← both hosts, independently

Sentry

renderer + Electron main
sendDefaultPii: false

The only external log sink — errors only, no alerting.

→ wilson-dev · → B2

GitHub Actions

rls.yml — pgTAP · smoke · vitest · e2e
backups.yml — 08:15 UTC

A `schedule` workflow only fires from the default branch — otherwise it is decoration.

→ builds both surfaces

Vercel

petal-studios/wilson
build:vercel + build:vercel:admin
backed by wilson-staging

← Electron renderer, direct

Google Drive

read-only, v0.1
tokens in rabbit-data/

The one external system a client reaches directly. Desktop only.

ARROWS THAT DELIBERATELY DO NOT EXIST

✗ No client → Anthropic. Ever, on either host.

✗ No desktop app → operator console.

✗ No cross-tenant read except `operator_workspace_summary()`.

✗ No `postgres_changes` subscriptions.

✗ No customer content → Backblaze B2.

✗ No O.T.T.E.R. or Notes content on any realtime topic, or in the company takeout.

The asymmetry is real. The desktop host carries an entire local data plane the web host does not have, and only the desktop reaches Google Drive. The operator console crosses the boundary at exactly two points — Edge Functions, and one direct `platform_audit` read gated by RLS alone.

One crossing, one guard. The console reaches every other tenant fact through service-role Edge Functions rather than widening RLS across tenants. The alternative — operator arms on a dozen policies — would put every company's rows one policy bug away from each other.

02 Two hosts · three environments · two build targets

handbook §3 · §4.1 · §5.1 · §5.2

> IN PLAIN TERMS
>
> One set of source code is baked into three different products: the Windows program, the website, and Petal Studios' private caretaker console. There are also three separate copies of the back office — one for experimenting, one for testing, one real one — so nobody develops against live client data. **The single most fragile thing in this section:** the website and the caretaker console live at the same web address, and the only thing keeping their logins apart is that each was built to remember your session under a different name. There is no second safety net behind it.

One React 19 + Vite + Tailwind 4 codebase, built three ways. Which build you are running determines which Supabase project you reach, which capabilities exist at all, and — for the operator console — which single `localStorage` key holds your session.

BUILD TARGETS — ONE TREE, FIVE SCRIPTS

`build`	`build:vercel`	`build:vercel:admin`	`build:admin`	`dev:admin`
`dist/`	`dist-vercel/wilson/`	`dist-vercel/wilsonadmin/`	`dist-web-admin/`	Dev server with both entries.
Feeds the Electron package. Emits **only** `index.html`.				

THE GATE `input: mode === 'admin' ? 'admin.html' : 'index.html'`

The entry is gated rather than always listed because plain `npm run build` feeds the Electron package — an unconditional second input would ship the platform operator console inside the desktop installer. **The desktop app has no code path to the console: it is not blocked at runtime, it is never built into that artifact.**

WHICH CLIENT HITS WHICH PROJECT

Local dev Supabase CLI linked here	**wilson-dev** eqjzmnvkrakroyqxfsvw
GitHub Actions smoke probe + Playwright	
Vercel beta /wilson + /wilsonadmin	**wilson-staging** rzkirvkotslbovzbsdfh
Shipped desktop app NSIS installer	**wilson-prod** rqyriuyldhovirbuievt

Migrations `0000`–`0029` and all twelve Edge Functions are deployed to **all three**.

CI's pgTAP job uses a **throwaway local stack**, not any of these — which is why the hosted `issue-session` smoke probe exists: the local stack issues HS256, so only a hosted probe catches an ES256 regression.

Staging and prod are dumped nightly to B2. Dev is not.

ANON KEYS ARE PUBLIC BY DESIGN

They ship inside the web bundle. Sign-ups are off, every table carries RLS, and the policy set is pinned by 37 pgTAP suites in CI. **The anon key identifies the project; it authorises nothing.** The *service-role* key is the secret, and it exists only inside Edge Function environments.

ONE ORIGIN, TWO SURFACES

beta.petalstudios.co

/wilson	/wilsonadmin
index.html → src/App.jsx	admin.html → src/admin/
wilson.dev.session	wilson.operator.session

THE MOST IMPORTANT FACT ABOUT THE CONSOLE — AND THE MOST FRAGILE

`localStorage` is scoped per **origin**, not per path. Both bundles read the same store. **The differing key string is the entire isolation mechanism** — set at build time by `__WILSON_SURFACE__`, not a runtime branch.

Any future code that reads a session key without going through `sessionStorage.js` reintroduces the leak. **There is no second mechanism to catch it.**

Two defences sit alongside it: supabase-js gets distinct `storageKey` values (`sb-wilson-operator` / `sb-wilson-app`) to namespace the navigator lock, PKCE slot and JWKS cache — *not* the isolation mechanism — and the admin surface **refuses the Electron safeStorage bridge entirely**, because that bridge is a single unkeyed slot that would defeat the key split.

ROUTING FACTS THAT LOOK LIKE TRIVIA AND ARE NOT

Vite names emitted HTML after its input. `dist-vercel/wilsonadmin/` contains `admin.html` and no `index.html` — which is why `vercel.json` rewrites `/wilsonadmin*` explicitly.

Security headers are scoped to `/wilsonadmin/:path*` only — `X-Robots-Tag`, `Referrer-Policy`, `X-Frame-Options: DENY`, `X-Content-Type-Options`.

No router. The shell renders every page and toggles `display`; on the web only, `navigateTo()` pushes `/wilson/<page>` and a `popstate` listener maps it back. Deep links, back and forward all work. Electron always boots `home`.

Install is `npm install --ignore-scripts`, not `npm ci` — Vercel's npm 11 rejects the npm-10 lockfile CI requires, and the web build needs no postinstall (Electron alone would pull ~100 MB).

03 Four tiers of authority

handbook §1 · §4.4 · §5.3 · §13.4

IN PLAIN TERMS

Four levels of authority. A **user** does their work. A **manager** sees rate cards and project history. An **admin** runs the company — invites people, changes roles, deletes projects. The fourth, **platform operator**, is Petal Studios itself and can create or destroy an entire company. Because that is so powerful, **it cannot be handed out from any screen anywhere** — someone has to type a command directly into the database. That means even a hijacked operator account cannot make more operators. The three "guards" underneath are the bouncers on those doors, and the document is explicit about which ones refuse when they are unsure and which ones let you through.

Lowest to highest. The fourth is not a bigger third — it lives on another surface, another session key, and another grant path entirely.

user

TIER 1 · ORDINARY MEMBER

One action in the matrix: `workspace.settings.read`.

Private content is private *from everyone* at this tier and above: `notes`, `note_subjects` and `otter_progress` carry **no admin bypass**.

manager

TIER 2 · TOOL-WIDE

Adds `rate_card.view`, `rabbit.history.view`, `rabbit.history.revert`, `project.create`, `member.profile.edit_others`. Bypasses project-level gating; sees change requests but cannot decide them.

Name collision, on purpose. `'manager'` is also a R.A.B.B.I.T. *project* seat. Different axis, same string.

admin

TIER 3 · COMPANY ADMIN

The eight gated actions: `admin.terminal.access`, `member.invite`, `member.remove`, `member.role.change`, `member.grants.change`, `project.delete`, `rate_card.edit`, `workspace.settings.write`.

A workspace can never reach zero active admins — `fn_ws_members_last_admin_guard`.

platform operator

TIER 4 · CROSS-TENANT

Petal Studios itself. Creates and destroys companies. Sign-in is email + password + TOTP — an operator has no company, so there is nothing for `resolve-login` to resolve.

CANNOT BE GRANTED FROM ANY UI

Writes to `platform_operators` exist in exactly three places repo-wide: one manual SQL runbook and two pgTAP fixtures. The highest privilege in the system cannot be escalated from a web session *even by someone already holding it*.

CLOSED **adminGuard**

1 bearer → 401
2 getUser → 401
3 claims: workspace_id + admin → 403
4 **live workspace_members row** → 403
5 MFA step-up → 403 mfa_required
lookup failed → **503 mfa_check_failed**

`WILSON_REQUIRE_ADMIN_MFA` ships **off** — a default-on gate would lock the owner out of her own Admin Terminal. Tracked as TPN-AUTH-003.

OPEN **memberGuard**

Same token + live-row shape, with **no role check and no MFA**. Any active member may use AI features; the live-row check is what stops a *deactivated* member spending money.

The rate limiter beneath it fails **open**, loudly. *A limiter is an abuse control, not an authorization control — and authorization has already run above it.*

CLOSED · HARD **operatorGuard**

no workspaceId in context
JWT claim **not required**
live platform_operators row only
no factor → 403 mfa_enrollment_required
lookup failed → 503 mfa_check_failed

Safe to be strict where the Admin Terminal cannot be: the surface is new, so no existing workflow breaks — **and it can delete a tenant.**

04 Postgres — the boundary, in detail

handbook §4.3 · §4.4 · §4.5 · §4.8

IN PLAIN TERMS

How the lock actually decides. In almost every case it asks two questions: *is this row your company's?* and *are you still an active member of that company?* The interesting part is the handful of places that deliberately break the pattern — personal notes and study progress that **not even a company admin can read**, and audit records kept deliberately unlinked so the proof survives after the thing it describes is deleted. The nightly clean-up jobs are listed too, along with the two record types that are **never** cleaned up, because they are the receipts.

Two canonical predicates cover almost every tenant-scoped table. What is worth a reader's attention is the short list of tables that deliberately deviate — each deviation is a decision with a reason, not an inconsistency.

CANONICAL READ — projects_select

```
deleted_at IS NULL
AND workspace_id = public.current_workspace_id()
AND public.has_active_membership(workspace_id)
```

CANONICAL WRITE — projects_insert

```
workspace_id = public.current_workspace_id()
AND public.has_active_membership(workspace_id)
AND public.current_app_role() IN ('admin', 'manager')
```

The third clause is specific to `projects` — creating a project is an admin/manager action. Child tables keep the two-clause form.

DELIBERATE DEVIATIONS — EACH WITH ITS REASON

if it looks asymmetric, it is — on purpose

`platform_audit`

Has a `workspace_id` column but **no FK**, and snapshots slug + name as text. A `workspace.teardown` certificate must still name the company after the row is gone. A migration post-condition fails the deploy if an FK ever appears.

`workspace_ai_keys · edge_rate_limits · storage_gc_queue`

Zero policies at all. Tenancy is enforced by the absence of any client grant. Service-role only.

`notes · note_subjects · otter_progress`

Tenancy **plus** `owner_id = auth.uid()` (`user_id` on `otter_progress`), with **no admin bypass**. A workspace admin cannot read another member's notes or study progress.

`otter_courses — personal tier`

No `current_app_role()` in the SELECT policy at all. Enforced by a post-condition that **fails the migration** if any SELECT policy on `otter_courses`/`otter_progress` ever mentions it.

`auth_attempt_log`

Operator-read, no membership requirement. Written **pre-authentication** — no workspace session exists yet.

`workspaces`

Keys on a membership subquery, **not** the JWT claim. It *is* the tenant row, and the Workspace Switcher needs to list every workspace you actively belong to — not only the current one.

LEARNED THE HARD WAY

Permissive RLS policies OR together. Adding a narrow policy alongside a broad `FOR ALL` one changes nothing — the old policy must be **dropped**. This cost Session 15 a critical finding that two independent passes caught.

TRIGGERS, BY JOB

AUDIT STAMPING

`fn_audit_touch()` — ~20 tables

POPULATE-WORKSPACE

`fn_populate_workspace_from_project`
`fn_populate_workspace_for_comment`
`fn_populate_workspace_for_project_member`
`fn_projects_populate_workspace (0029)`

IDENTITY PINS

`fn_ws_members_prevent_self_role_change`
`fn_otter_pin_course_identity`
`fn_otter_pin_subject_identity`
`fn_otter_pin_progress_identity`
`fn_otter_editor_grant_workspace`

A disallowed course-visibility change is **silently reverted** — the UI must trust the returned row, never what it sent.

GUARDS

`fn_ws_members_last_admin_guard`
`fn_workspaces_client_guard`
`fn_workspaces_delete_guard (0029)`
`fn_soft_delete_stamp`

CAPTURE

`fn_edit_history_capture` — 13 tables
`fn_file_events_capture`
`fn_app_events_stamp`
`fn_files_gc_enqueue`

BROADCAST · STAFFING · STATE

`fn_realtime_broadcast` — 10 tables
`fn_workspace_realtime_broadcast` — 5
`fn_projects_auto_staff`
`fn_otter_cr_review`

SHARED CONTRACT Every capture trigger carries `EXCEPTION WHEN OTHERS → RAISE WARNING`. **An audit failure must never abort the write it audits** — including a workspace CASCADE.

REALTIME — BROADCAST, NEVER postgres_changes

> `postgres_changes` authorizes each event by evaluating the subscriber's SELECT policy against the **NEW row**. Under soft-delete policies, setting `deleted_at` makes that row invisible — so the single event collaborators most need, *"this was just trashed"*, would be silently withheld.

rabbit:project:{id}
10 tables · 0016
`can_read_project_topic()`
`INVOKER, ≡ projects_select`

rabbit:workspace:{id}
5 tables · 0018
`can_read_workspace_topic()`

Merge is LWW per field. Fields with an in-flight local write — the *pending-field set* — keep the local value; an `updated_at` stale guard drops older rows; `SUBSCRIBED` always fires a debounced refetch, including the first join, to close the missed-events window.

Deliberately **never broadcast**: `otter_*` and `notes` / `note_subjects` — the workspace channel hands full row payloads to every subscriber.

pg_cron — FIVE NIGHTLY SWEEPS

04:43	wilson-purge-edit-history	90d
04:47	wilson-purge-soft-deleted — 7 tables, cascading	30d
04:51	wilson-purge-app-events	90d
04:55	wilson-purge-otter-trash	30d
04:59	wilson-purge-rate-limits	1d

Two streams have no purge job, and that is a compliance position: `file_events` (the `'purged'` rows *are* the deletion certificates) and `platform_audit` (a teardown certificate must outlive the company it describes). `storage_gc_queue` is drained on an admin's click, never by cron.

05 Edge Functions ×12

handbook §4.6 · §5.4 · §5.5

> IN PLAIN TERMS
>
> Twelve small trusted programs. The database can protect data, but it cannot create a user account, keep a secret, or act on two companies at once — so these do those jobs. They hold a master key that ignores the locks, which is exactly why each one is written to look up your real record before doing anything. They are grouped below by how strict their door is.

They exist only for the operations RLS cannot express: minting users, holding secrets, crossing tenants. Grouped by the guard that fronts them — the guard is the interesting part.

ALL TWELVE `verify_jwt = false` — The Edge gateway's built-in verification only supports HS256; these JWTs are ES256, so it rejects them with `UNAUTHORIZED_UNSUPPORTED_TOKEN_ALGORITHM` before any function code runs. Each function validates the caller itself with `admin.auth.getUser(token)`. **Do not "fix" a function by turning `verify_jwt` back on — it will break, not harden.**

NO GUARD — PUBLIC, PRE-AUTH

resolve-login

Username → email. A constant-time floor of **180 ms** wraps every reply path and the body is always `{exists, email}` — neither timing nor shape distinguishes a hit from a miss. On a miss the *client* still signs in against `__miss+<uuid>@invalid.local` so the code path is identical too.

Two failure paths return a miss with **no** `auth_attempt_log` row: a members-query error and an `admin.getUserById` failure.

provision-workspace

Creates company + first admin atomically via `provision_workspace_and_admin`, plus up to 19 initial invites. Called by `NewCompanyWizard`, which deliberately does **not** auto-sign-in — it hands `{username, slug}` back so the login screen opens pre-filled.

INLINE getUser

issue-session

It does not mint a token. Its entire job is to write `app_metadata.workspace_id` via the admin API, so that the client's *next* `refreshSession()` causes GoTrue to mint a new token in which the hook re-derives every claim. Called at sign-in and on every workspace switch.

CLOSED requireWorkspaceAdmin — adminGuard

admin-create-user

Member + **show-once** password; optional synthesized email.

admin-reset-password

Rotate a member's password, show-once.

admin-set-active

Deactivate/reactivate + GoTrue ban + best-effort global sign-out. Last-admin guarded.

admin-user-security

Read-only posture: email, last sign-in, ban state, MFA factors.

storage-gc

Queue drain · orphan scan · avatar sweep. **Admin-invoked, never cron** — the click is TPN's second authorization factor. Emits `WIL-3003` / `WIL-3004`.

invite-member — inline guard, claims + live row

Invite by email; membership row created with `onboarded_at` null, which is what makes `NewUserWelcome` appear.

Known gap: no MFA step-up, unlike its sibling — and it can create an `admin`, so an admin with a verified factor can mint another admin from an `aal1` session.

OPEN requireActiveMember — memberGuard

ai-proxy

The single path to Anthropic, from every AI feature on both hosts. Drawn in full in section 06.

CLOSED · HARD requirePlatformOperator — operatorGuard

operator-workspaces

`list · create · rename · suspend · restore · teardown`

`list` pages `operator_workspace_summary()` — **the single cross-tenant read in the system** — and includes suspended companies, which the member-facing policy hides. Teardown is drawn in section 09.

operator-ai-keys

`set · clear` — `there is deliberately no get`

`set` validates with a live 1-token call to Anthropic; only a 401/403 counts as invalid — anything else is inconclusive and the key is stored anyway. AES-256-GCM, `base64(iv[12] ‖ ct ‖ tag[16])`, fresh IV per record, key from `WILSON_AI_KEY_SECRET` which must decode to exactly 32 bytes. Certificates `WIL-7010` / `WIL-7011` carrying only the hint.

WHY ENCRYPT A TABLE THAT IS ALREADY SERVICE-ROLE-ONLY WITH ZERO POLICIES?

Because the nightly `pg_dump` goes **off-platform** to Backblaze B2 with 90-day retention. A plaintext column would copy every tenant's Anthropic spending credential into a 90-day off-platform archive. **The ciphertext goes into the dump; the key that opens it does not.**

Vault was rejected: CI's pgTAP job runs a local stack that cannot run on the development machine (no Docker), so a Vault dependency inside the migration chain could not be verified before it either passed or broke every job at once.

CONSEQUENCE TO KNOW

Rotating `WILSON_AI_KEY_SECRET` **orphans every stored company key**. The ciphertext becomes undecryptable, `ai-proxy` fails soft to the platform key, and the console still shows the old hint. There is a `key_version` column but no version-conditional key selection — rotation needs a re-encryption pass.

Show-once credentials. WILSON stores no password anywhere. `CredentialsPopup.jsx` holds the plaintext in component state only, has no backdrop-click / Escape / X dismissal, and requires a second confirmation to close without copying.

06 The `ai-proxy` seam

handbook §6

> IN PLAIN TERMS
>
> Every AI feature in all three tools goes through one single door. The app never holds the AI account key — it asks WILSON's own server, and that server talks to Anthropic. That means a company's AI spending credential can never leak out of a laptop, and every AI request is logged with what it cost. A company can supply its own AI key; if anything goes wrong reading it, the system quietly falls back to Petal Studios' key, **because whose key is used is a billing question, not a security one**.

No Anthropic API key ever reaches a client, on either host. That is verifiable, not asserted: `x-api-key` and `ANTHROPIC_API_KEY` appear nowhere under `src/` or `electron/`, and the app purges two legacy `localStorage` key slots on every launch. There is **no direct-to-Anthropic fallback anywhere** — a second code path is precisely how the pre-proxy divergence bugs happened.

CLIENT — BOTH HOSTS

callAI()

`src/cloud/aiProxy.js`
→ `POST /functions/v1/ai-proxy`

Accepted fields: `model`, `max_tokens`, `messages`, `system`, `tools`, `betas`, and `tool` — WILSON's own attribution tag, ≤40 chars, logged and **never forwarded upstream**.

`anthropicStream.js` reassembles SSE back into the classic non-streaming message object — which is why O.T.T.E.R.'s single wrapper changed internally and none of its six call sites did.

EDGE FUNCTION — `service_role`

1 · requireActiveMember

Token claims *plus* a live `workspace_members` re-check. A deactivated member gets `403 forbidden` even with an unexpired token — this is a **spend** endpoint.

2 · fn_rate_limit_hit — fails OPEN

Bucket `ai-proxy`, subject = workspace, 60 s, `AI_PROXY_RPM` default 60. **Fixed** window, not sliding — up to 2× the limit can pass across an edge.

3 · the key seam

`workspace_ai_keys → decrypt`
`↳ any failure → platform ANTHROPIC_API_KEY`
`↳ neither → 501 ai_not_configured`

A tenant key is a billing preference, not an authorization boundary. Deliberately 501 and not 503: every client retry loop treats 429/503/529 as transient, so a 503 would burn ~21 s of pointless retries before the honest message surfaced.

4 · upstream body is a whitelist reconstruction

Not a passthrough — clients cannot smuggle extra fields.

5 · telemetry, fire-and-forget

`EdgeRuntime.waitUntil` → `app_events`. `WIL-6001` success / `WIL-6002` failure, with model, tool, token counts, `stop_reason` and `key_source`. Never blocks the response.

ANTHROPIC

api.anthropic.com

`stream: true` — always

Regardless of what the client asked for. Edge Functions must emit a response within **150 s**; O.T.T.E.R. generations regularly run longer, so a synchronous proxy would occasionally 504 and lose an entire generation. With SSE the first byte is immediate and the 400 s wall clock governs instead.

`claude-sonnet-4-20250514`
`outlines · content · deck outlines · image prompts · agent turns · Validator · heavy intake`

`claude-haiku-4-5-20251001`
`companion · theme colours · rewrites · quiz · light intake + classifier · key validation`

Each call site hardcodes its model; there is no shared default constant. Retry policy is **deliberately inconsistent by caller**.

THE IN-STREAM ERROR TRAP

Anthropic can return HTTP 200, then emit an SSE `error` event mid-generation, and still close the stream *normally*. There is no non-2xx status to key off. Both sides trap it — the proxy's usage sniffer flips telemetry to `WIL-6002`, and the client reassembler throws with a status mapped from the Anthropic error type. **Anything that logs "completed" on stream close must sniff for in-stream errors.**

06b Sequences — who calls whom, in what order

handbook §4.2 · §4.5 · §4.6 · §9

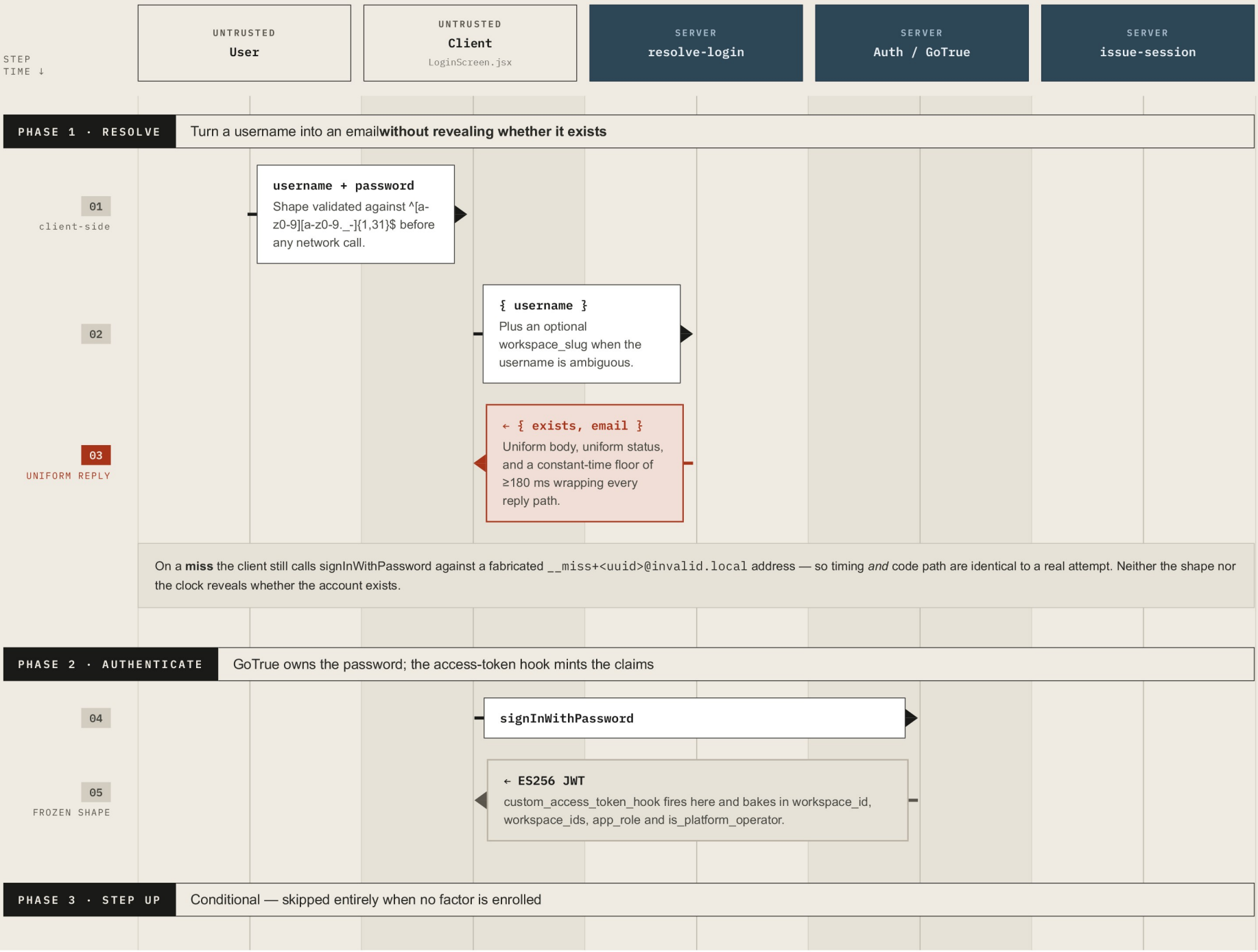
IN PLAIN TERMS

The moments where several systems have to cooperate, broken into numbered phases. Participants run across the top; time runs down. Each arrow is one call, a dashed arrow is the reply, and **the step marked in red is the one carrying the security property** — if you only look at one line per diagram, look at that one. **Logging in is the one worth reading closely:** it behaves identically whether or not the username exists, so the login screen cannot be used to discover who works at a company.

HOW TO READ — a call — ← the reply — ↷ in place — no call leaves — 03 — carries the security property — one lane per participant

F1 — LOGIN

handbook §4.2 · five phases, twelve calls


STEP
TIME ↓
UNTRUSTED
User
UNTRUSTED
Client
LoginScreen.jsx
SERVER
resolve-login
SERVER
Auth / GoTrue
SERVER
issue-session
PHASE 1 · RESOLVE
Turn a username into an emailwithout revealing whether it exists
01
client-side
username + password
Shape validated against ^[a-z0-9][a-z0-9._-]{1,31}$ before any network call.
02
{ username }
Plus an optional workspace_slug when the username is ambiguous.
03
UNIFORM REPLY
← { exists, email }
Uniform body, uniform status, and a constant-time floor of ≥180 ms wrapping every reply path.
On a miss the client still calls signInWithPassword against a fabricated __miss+<uuid>@invalid.local address — so timing and code path are identical to a real attempt. Neither the shape nor the clock reveals whether the account exists.
PHASE 2 · AUTHENTICATE
GoTrue owns the password; the access-token hook mints the claims
04
signInWithPassword
05
FROZEN SHAPE
← ES256 JWT
custom_access_token_hook fires here and bakes in workspace_id, workspace_ids, app_role and is_platform_operator.
PHASE 3 · STEP UP
Conditional — skipped entirely when no factor is enrolled

06

07

PHASE 4 · CHOO

08

09

10

SEATS ONLY

PHASE 5 · RE-M

11

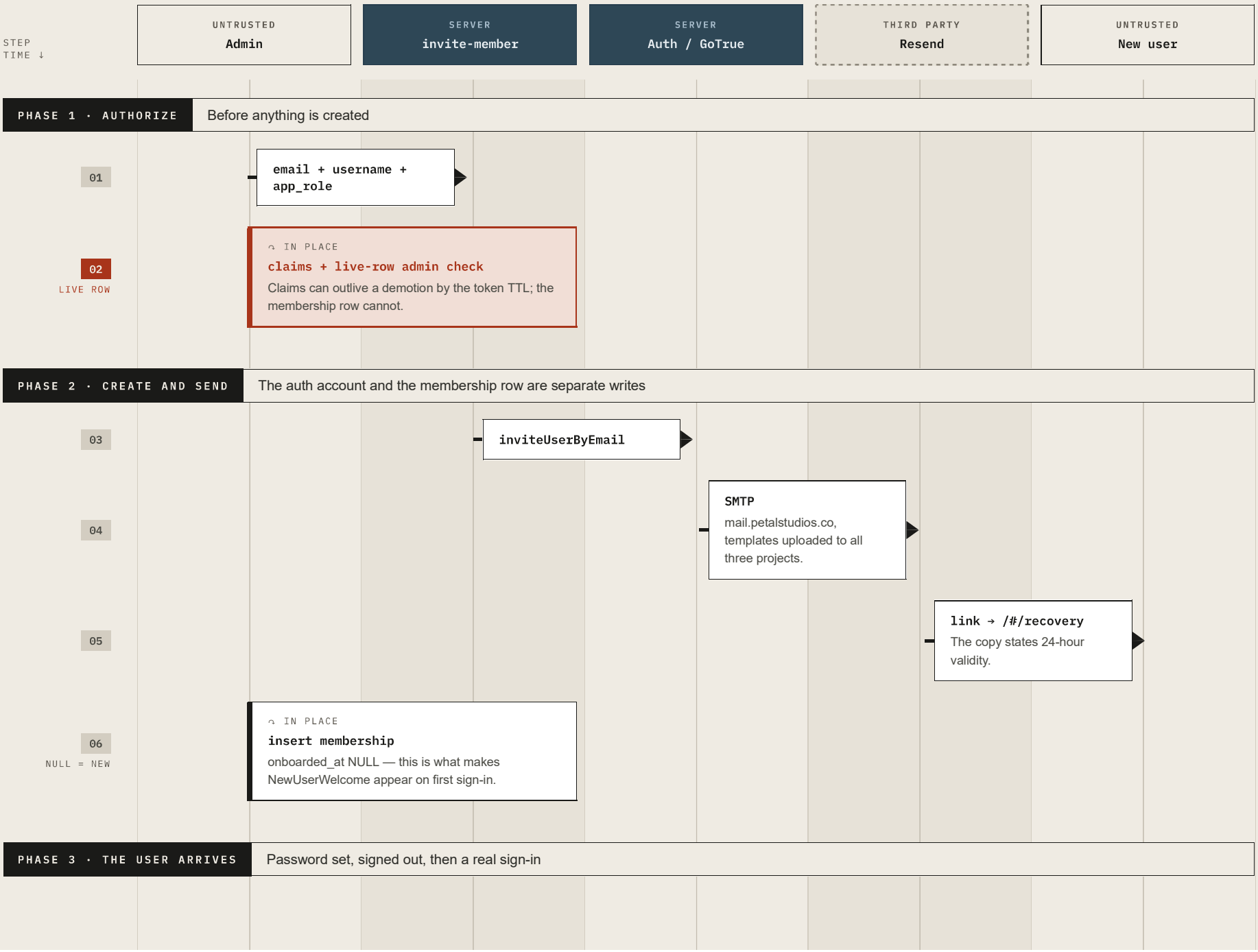
12

CLOSED Most branches write an `auth_attempt_log` row, readable only by platform operators — but **two failure paths return a miss with no row at all**: a members-query error and an `admin.getUserById` failure.

- Login is **username-first**: a user types a workspace username, never an email. An operator has no company, so there is nothing to resolve against — which is why the operator console signs in with email + password + TOTP instead.
- A username that collides across two workspaces makes sign-in **unreachable**: the resolver treats two matches as a miss unless a slug disambiguates, and the login screen has no slug field. Tracked.

F2 — INVITE AND ONBOARDING

handbook §4.6 · §9 · three phases, ten steps


STEP
TIME ↓
UNTRUSTED
Admin
SERVER
invite-member
SERVER
Auth / GoTrue
THIRD PARTY
Resend
UNTRUSTED
New user
PHASE 1 · AUTHORIZE
Before anything is created
01
email + username + app_role
02
LIVE ROW
IN PLACE
claims + live-row admin check
Claims can outlive a demotion by the token TTL; the membership row cannot.
PHASE 2 · CREATE AND SEND
The auth account and the membership row are separate writes
03
inviteUserByEmail
04
SMTP
mail.petalstudios.co, templates uploaded to all three projects.
05
link → /#/recovery
The copy states 24-hour validity.
06
NULL = NEW
IN PLACE
insert membership
onboarded_at NULL — this is what makes NewUserWelcome appear on first sign-in.
PHASE 3 · THE USER ARRIVES
Password set, signed out, then a real sign-in

07

08

09

10

→ stamps onboarded_at.

- An admin-created user may have **no real email** — one is synthesized as `wilson.<workspace-prefix>.<username>@mail.petalstudios.co` purely to satisfy GoTrue's shape requirement. Consequence: forgot-password is unavailable for those accounts and an admin must reset instead; an `email_synthesized` flag lets the UI say so.

FROZEN `invite-member` performs **no MFA step-up** while its sibling `admin-create-user` does, and it can create a member with `app_role: 'admin'` — so an admin holding a verified factor can mint another admin from an `aal1` session. Tracked.

F3 — REALTIME: BROADCAST AND MERGE

handbook §4.5 · four stages

1 A WRITE HAPPENS

any of 15 tables

INSERT · UPDATE · DELETE
Ordinary client write, through PostgREST under the caller's own token.

AFTER trigger
`fn_realtime_broadcast` — 10 tables
`fn_workspace_realtime_broadcast` — 5

`realtime.broadcast_changes`
Skips cleanly when absent, and never aborts the write it follows.

↓

2 ONTO ONE OF TWO TOPICS

scope decides who could ever hear it

`rabbit:project:{id}`
projects · phases · assets · tasks · files · comments · task_dependencies · task_links · asset_versions · project_members

`rabbit:workspace:{id}`
projects · workspace_members · project_members · tasks *(only when assignee/reviewer set)* · assets *(only trash/restore, or a name/phase change)*

↓

3 JOIN AUTHZ — ONCE, AT JOIN TIME

not per event

`can_read_project_topic()` ≡ `projects_select`
`can_read_workspace_topic()`

Authorisation is evaluated **when the client joins the channel**, not on every message. `fn_try_uuid()` guards the topic-suffix cast, so a hand-crafted topic string cannot error a policy.

↓

4 THE CLIENT MERGES

realtimeMerge.js — pure, no React, no adapter

`pending fields keep local`
Fields with an in-flight local write, counted per `table:id`, are never overwritten by an incoming row.

`stale updated_at dropped`
An incoming row older than the local one is discarded outright.

`SUBSCRIBED → debounced refetch`
Fired on *every* join, including the first, to close the missed-events window.

FROZEN **Not** `postgres_changes`. It authorizes each event against the **NEW row** of every UPDATE — so under soft-delete policies, setting `deleted_at` makes the row invisible, and the one event collaborators most need ("this was just trashed") would be silently withheld.

CLOSED Deliberately **never broadcast**: `otter_*` and `notes` / `note_subjects` — the workspace channel hands full row payloads to every subscriber.

- *Operational gotcha:* on a hosted project whose Realtime tenant has never been active, `realtime.messages` has no partitions and `realtime.send()` **silently drops rows** until the first websocket connection activates the tenant.

07 Three tools, three shapes

handbook §13.1 · §13.2 · §13.3

IN PLAIN TERMS

The three tools share a login, a staff list and a shell — and almost nothing else. **D.O.G.** is a conveyor belt: documents in, slide outlines out. **O.T.T.E.R.** is a library with a borrowing system: courses can be private, shared, or the official company version, and changing an official one works like proposing an edit for approval. **R.A.B.B.I.T.** is a funnel: creative documents go in, and a full production plan — phases, assets, tasks, budgets, schedule — comes out. Drawing them as one shape would be a lie.

D.O.G. v0.514

Deck Outline Generator — *a conveyor belt*

Source files + project context

`assemble messages → callAI()`

`parseSlideContent()`
TITLE · SUBTITLE · LAYOUT STRUCTURE · COPY · VISUAL STYLING · REQUIRED ASSETS · COMPONENT GEOMETRY

`correctGeometryIconOrder()`

Slide tabs → exports

`{deck}_DECKOUTLINE.md`
`{deck}_VIS_DECKOUTLINE.md`
`{deck}_IMG_PROMPTS.md`
`{proj}_VIS_ASSETS_{NN}_{name}.{ext}`

Treat the three canonical filenames and their structure as a **contract** — downstream tooling consumes them.

> **The COMPONENT GEOMETRY JSON has no in-app renderer.** Its consumer is an external Google Slides extension. `LayoutVisualizer.jsx` deliberately re-derives its own approximate layout and *never* reads the AI's x/y/width/height. Do not "fix" it without knowing that.

Canvas **720 × 405 pt**; safe area x ∈ [36, 684], y ∈ [30, 370]. Full-deck generation loops up to **3 continuations** on `stop_reason: max_tokens` — a continuation loop, not an error retry. **D.O.G. is not part of the agent system.**

O.T.T.E.R. v0.3.1

Courses — *a library with a borrowing system*

```
course
  └ subject — may be a stub
      └ section
          └ lesson
```

Plus five whole-document reference blobs per course: `hotkeys`, `functions`, `nodes`, `reference_urls`, `corrections`.

`personal`
Owner only. **No admin bypass.** An admin can see it *exists*, never its contents.

`shared`
Every active member reads it.

`company_standard`
Admin only, in both directions; at most one per topic. Using one **forks a personal copy**, so the official version stays pristine and admin edits never change a course underneath someone mid-study.

> **Approving a change request APPLIES it.** `otter_cr_apply()` is the only path — a bare status flip raises. It archives the target first, then applies subjects **additively**: update by slug, insert when absent, **never delete**. One approval must not silently strip lessons from everyone's official course.

Apply does **not** touch the five reference documents — their merge semantics live in client JavaScript, and overwriting them would violate additive-only. The approve dialog says so.

Quiz scores and Validator findings are never persisted. Session state only.

R.A.B.B.I.T. v0.1.0

Resource Allocation, Budgeting & Breakdown Intake Tool — *a funnel*

`is_core_definer` files

`extract text`
txt · md · fountain direct · docx via mammoth · pdf via local route · pptx via JSZip

`detect document kind`
explicit → extension hint → regex → Haiku classifier

`1 of 9 chunkers → worker pool ×3`
+ persona block: executive / creative / technical

`fuzzy-dedup merge → review`

accept → phases → assets → tasks

> **Failure behaviour is deliberately asymmetric.** Individual chunk failures are tolerated; if **every** chunk fails the pool throws. An earlier version had a fake-success path where all chunks failed, produced an empty breakdown, and reported "Intake complete".

Three adapters, one interface — `supabaseAdapter`, `localServerAdapter`, `googleDriveAdapter` — and nothing may bypass it. Scenes, shots, levels, experiences and milestones are **local-only**; their Supabase methods throw with a stated reason.

A fifth role family lives here at project level — `manager` / `reviewer` / `member` — a different axis from the workspace tiers.

LOCKSTEP INVARIANT

`src/permissions/projectRoleMatrix.js` mirrors the SQL helpers `can_write_project()`, `can_comment_project()` and `can_manage_project_roster()` from migration 0013. **Any change to one must ship with the matching change to the other.** The migration's `COMMENT ON FUNCTION` points back at the JavaScript file by name, closing the loop from both ends. The same holds for `roleMatrix.js`, which is mirrored 1:1 in a unit test that fails on any mismatch in either direction.

08 Screen schematics

handbook §13.1–13.4 · §5.6

IN PLAIN TERMS

Where things sit on each screen — which panes exist, what goes in them, and in what order. These are **arrangement diagrams, not pictures of the product**: grey bars stand in for content, and only the two colours the handbook actually names are used. They also carry the one structural quirk that explains a whole class of bugs: **every page is loaded at all times and simply hidden.** Switching pages only changes which one is visible — so a keyboard shortcut written inside one tool is listening while you are in a different tool, and each one has to check where you actually are.

DATED

The home screen and the three tool screens below are drawn from **screenshots of a build taken before the multi-user migration** — the chrome, layout and copy are the product's own, but they predate this handbook. Where multi-user changed a screen, the change is stated in red beside it rather than drawn, because that state has not been seen. **W6 and W7 have no screenshot** and remain arrangement-only.

rabbit | home | dog | otter | dashboard | project-manager | rate-card | team-members | admin-terminal | settings | help

all 11 mounted · one shown · display toggled · ~2.1 s five-phase transition

TWO CONSEQUENCES THAT HAVE EACH ALREADY CAUSED A BUG

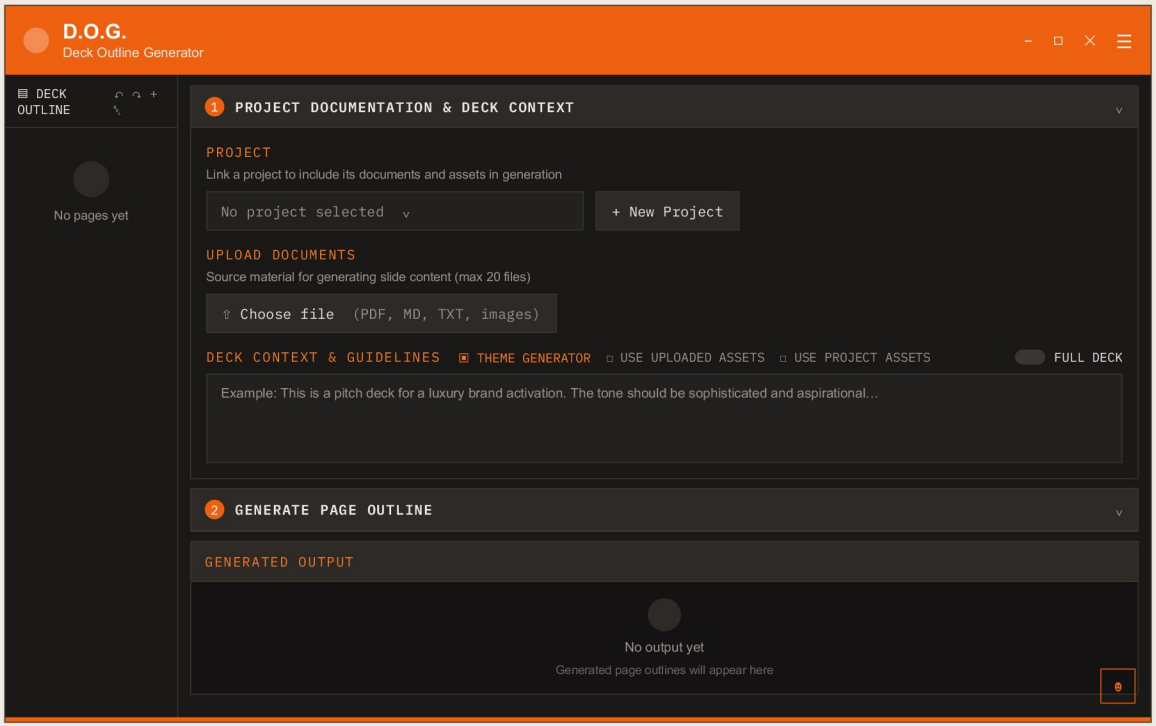
1 · Every page's effects run everywhere. A window-level `keydown` listener inside a tool fires while the user is in a different tool. Gate on `currentPage` — O.T.T.E.R.'s sidebar-collapse shortcut does exactly this. *One ungated listener remains: Space-to-search.*

2 · Every page's data hooks run for every user. `AdminTerminalPage` renders for everyone at every launch, so a page-level hook would fire a `workspace_directory` RPC for every user. Keep the gated component cheap; push the expensive hook into a child that mounts *after* the role check, and hand each section an `isActive` prop.

`PageShell.jsx` is unreferenced — the shell reimplements it inline — and `ToolShell.jsx` is an explicit pass-through stub.

The title bar is Electron-only. `frame: false`, `contextIsolation: true`, `nodeIntegration: false`, a preload bridge, and Ctrl+R / F5 re-mapped to "reset zoom" rather than reload.

The undo toast is bottom-centre, 8 s, pauses on hover, and targets its exact history entry so a click and Ctrl+Z cannot double-fire.

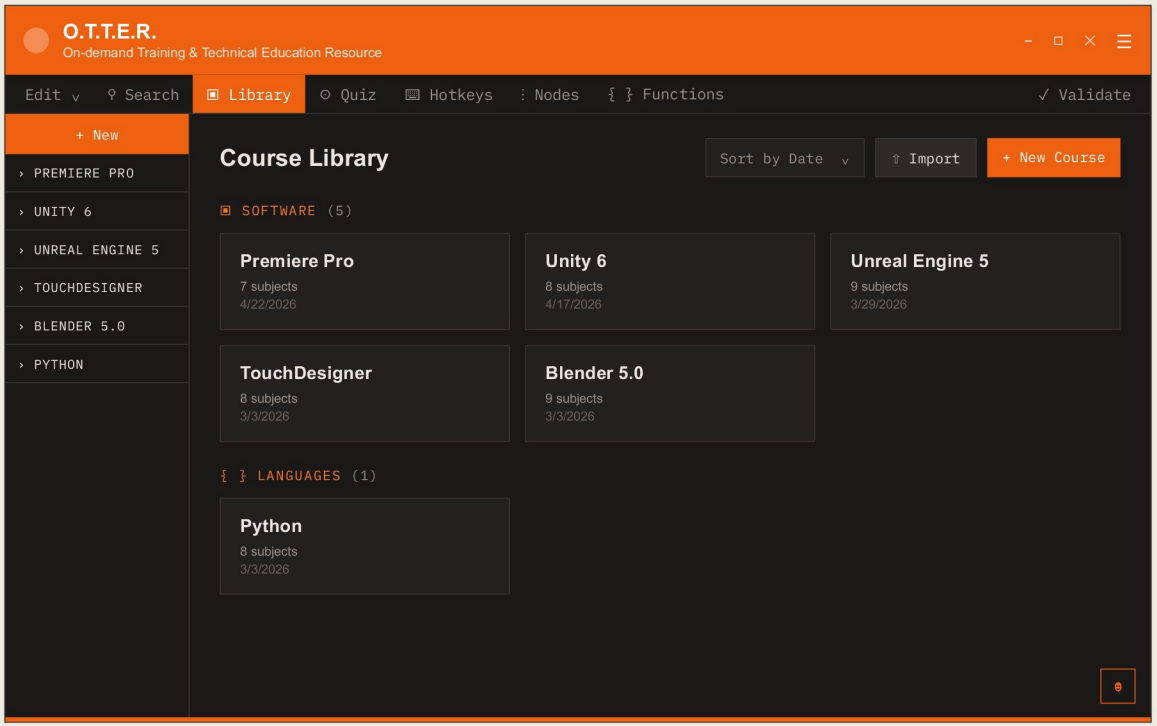


A narrow left rail of generated page thumbnails beside **one scrolling column of numbered, collapsible stages** — not a three-pane editor. Stage 1 gathers project documentation and deck context; stage 2 requests a page; Generated Output sits beneath.

> CHANGED SINCE THIS BUILD — MULTI-USER
>
> Per-user API keys are gone, so the red “API key required — set it in Settings” line no longer reflects how keys work. Every AI call now goes through `ai-proxy` with the workspace or platform key, and a missing key surfaces as `501 ai_not_configured`.

The context stage carries the three generation switches — **Theme Generator · Use Uploaded Assets · Use Project Assets** — and the **Full Deck** toggle, which selects the up-to-3-continuation path described in section 07.

Uploads cap at **20 files**. In local mode every upload is read as a data URL so `content` is always present; in cloud mode the Supabase adapter strips attachments and **throws** rather than losing them silently.



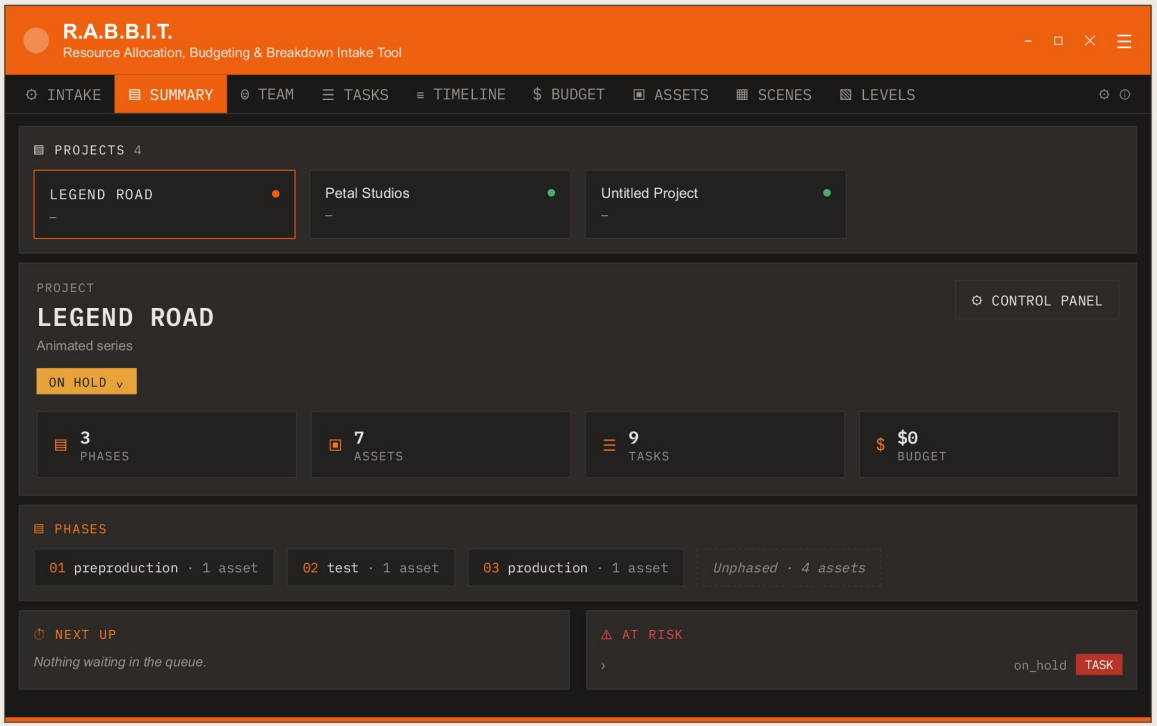
Orange app header, then a single nav row — **Edit · Search · Library · Quiz · Hotkeys · Nodes · Functions** with **Validate** pinned right — over one course sidebar and the main pane.

The Library groups courses by kind (**Software** / **Languages**), which is the same split that makes the `hotkeys` view dual-purpose: Keyboard Shortcuts for software courses, Functions Reference for coding-language courses.

Sidebar collapse is `Ctrl/Cmd + \` — gated on `currentPage === 'otter'`, one of only two listeners in the app that does gate.

CHANGED SINCE THIS BUILD — MULTI-USER

A second sidebar for lessons, tier chips (**personal / shared / company_standard**), the “Private” badge on courses an admin can see but not read, the trash filter and the **Requests** tab all arrive with multi-user. `requests` and `validator` are conditionally *mounted*, so they do not appear at all for users without the capability.



The view tabs sit **directly under the orange bar, above the project selector** — Intake · Summary · Team · Tasks · Timeline · Budget · Assets · Scenes · Levels. Project switching is a horizontal strip of **project cards with status dots**, not a dropdown.

Summary is a stats dashboard: project header with its status pill and Control Panel, four stat tiles, the folder row, phases as chips, Next Up / At Risk, and the budget snapshot. **Phase numbering and the "Unphased" bucket** are the same hierarchy the intake pipeline writes: phases, then assets, then tasks.

The FOLDER row is the `files_dir` surface from section 09 — "using default location" until a relink or an explicit change re-homes it, with the Reset control alongside.

> CHANGED SINCE THIS BUILD — MULTI-USER
>
> The project switcher moves into the header, joined by **presence chips** and the **LIVE / SYNC / SYNC ERR** pill riding the realtime channels — none of which exist in this build. Scenes, Levels and Experiences become *toggleable* per project, and remain **local-only**: their Supabase adapter methods throw with a stated reason.

W6 — ADMIN TERMINAL

SCHEMATIC · §13.4

app_role = admin

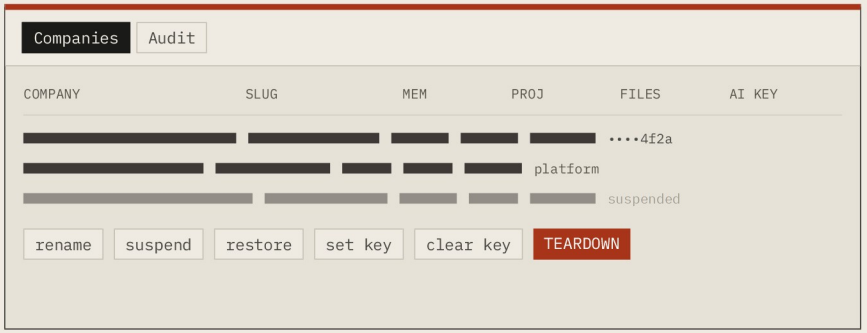
Each section **lazy-fetches on first activation** (`isActive`), never at page mount — because this page is rendered for every user at every launch.

Company hosts **Workspace Takeout**; Diagnostics hosts **Storage Cleanup**. Requests is the O.T.T.E.R. change-request queue — admin work in the admin place.

W7 — OPERATOR CONSOLE

SCHEMATIC · §5.6

/wilsonadmin



No router, no tools, no pet. Sign-in is **email + password + TOTP**.

Audit is the one place the console reads the database **directly** with the operator's own client — the latest 200 `platform_audit` rows, gated purely by RLS. Writes are impossible: one SELECT policy, all write privileges revoked, and a migration post-condition that raises if that ever stops being true.

> "Not a platform operator" is a distinct screen reached only *after* a fully successful sign-in — which, like the MFA stage, inevitably discloses that the credentials were valid. Wrong email and wrong password are unified into one generic error; this is deliberately not unified with them.

09 Lifecycles — delete, relink, teardown, review

handbook §12.2 · §12.3 · §12.4 · §5.4 · §13.2

IN PLAIN TERMS

Four processes where the *order of operations is the safety feature*. Deleting something never destroys it immediately — it goes to a bin for 30 days, and when it is finally destroyed a permanent receipt is written that **outlives the thing it describes**, so you can prove a file was destroyed even after the project and the company are gone. Destroying a whole company follows an eight-step sequence that cannot be shortened: you have to write down what you are about to delete *before* you delete it, because afterwards there is no way to know what belonged to whom.

DELETE → TRASH → PURGE → CERTIFICATE

Live row → **Trashed** `deleted_at` set → **Hard delete** cron 04:47 · 30d

`soft_delete_row()` ⇄ `restore_soft_deleted()` — RPCs only: a plain UPDATE cannot, because Postgres re-checks the SELECT policy on both sides

THE CERTIFICATE

`file_events · 'purged'`

No FKs on `file_id` / `project_id` — deliberately, **so the certificate outlives its subject**. The read policy carries a workspace-admin arm so proof of deletion stays readable after the project is gone.

THE BLOB

`storage_gc_queue → storage-gc`

Admin-invoked, never cron. TPN wants dual authorization on destruction and a human clicking a stated confirm *is* the second factor. Emits `WIL-3003` / `WIL-3004`.

Three jobs per run: **queue drain** (re-checks that no `files` row, live *or* trashed, still references the path, so a restorable file's blob is never destroyed), **orphan scan** (5000-object budget, only unreferenced objects older than 24 h, failing **closed** on tenancy), and **avatar sweep** (paged member reads — the unpaged version deleted *current* avatars past the 1000-row cap).

Vocabulary is a CHECK constraint, not a convention: `uploaded`, `moved`, `relinked`, `trashed`, `restored`, `purged`. Cloud path changes emit `moved`; local relink emits `relinked`.

RELINK — FIND, PREVIEW, APPLY

Modelled on ShotGrid/Blender: point WILSON at the new folder and it re-finds the moved files itself. `local_server` only, but the matcher is provider-agnostic and unit-tested in isolation.

`1 · exact`
Basename matches, case-folded. Duplicate disk names tiebreak on size, else **ambiguous**.

`2 · strong`
Sanitized name **and** size. Run as a full pass before rung 3, so a size-mismatched row cannot steal a candidate a later row would match strongly.

`3 · name`
Name only. Zero hits → **unmatched**; more than one → **ambiguous**.

`files_dir` IS A BASE-CHANGE, NOT A ONE-OFF

Applying a relink makes that folder the project's files home **for future uploads too**. Four guards: the dialog discloses it *before* the write; Files & Storage surfaces a Reset control; a **409** refuses if the recorded home exists but is merely *offline*; a second **409** refuses if the change would strand any file.

403 for any folder not picked through the OS dialog. `rabbit:pick-directory` recording a pick into `userAuthorizedDirs` *is* the authorization — it is the only way a folder ever enters that set.

ORDER **WORKSPACE TEARDOWN — THE ORDER IS THE DESIGN**

Reading it in sequence is the only way to understand why it cannot be simplified.

1 Snapshot the workspace row first

Name and slug copied **as text**. That snapshot is what keeps the certificate meaningful after the row is gone.

2 Require the slug typed back

Else **400** `confirmation_mismatch`.

3 Collect every rabbit-files path

From **both** `files` and `storage_gc_queue`, paged with `.range()` in 1000-row chunks. Paging is not optional: PostgREST caps un-ranged reads at `max_rows` **even for service_role**.

4 Refuse foreign paths

Only `projects/{project_id}/…` whose project belongs to *this* workspace. `files.storage_path` is client-writable and the sweep runs as service_role — a member could otherwise point a row at another tenant's key and have teardown delete it. Rejected paths produce a refusal certificate, `WIL-7008`.

5 Delete blobs in batches of 100, certificate each batch

`WIL-7006`, emitted in 40-path chunks because the audit `context` column has an 8000-character CHECK. `blobs_removed` counts what the bucket actually returned, not the batch size — **a certificate must never claim a destruction that did not happen**.

6 Delete the workspace row

This fires the CASCADE.

7 Drop the queue rows — AFTER the cascade

The `files` CASCADE **re-enqueues** one `storage_gc_queue` row per file; clearing first would leave those undrainable.

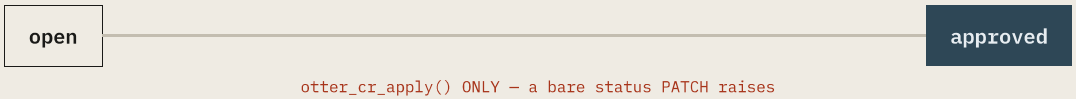
8 Write the final certificate

`workspace.teardown` — `WIL-7005`, severity critical, with found / removed / missing / failed / rejected counts.

Why collection must come first: after the CASCADE there is no way to discover which blobs belonged to the tenant, and `storage-gc` orphan scan fails closed because the rows it resolves through are gone.

Named limitation. Teardown removes the tenant, not the people. A user whose only membership was in that company keeps an `auth.users` row and can still authenticate; they simply resolve to no workspace. Deleting those identities would be a destructive act on accounts the operator did not create. What is missing is the *reporting*.

CHANGE-REQUEST STATE MACHINE — O.T.T.E.R.



`otter_cr_apply()` ONLY — a bare status PATCH raises

`open → rejected`	Reviewer. Note optional.
`open → changes_requested`	Reviewer. **Note required** — enforced by both the trigger and a CHECK.
`open → withdrawn`	Proposer. Terminal.
`changes_requested → open`	Proposer resubmits. `revision += 1`; reviewer stamps cleared.
`changes_requested → rejected`	Proposer accepts the decision. `acknowledged_at` set; the decliner stays reviewer of record.

THE CONSENTED REVIEW WINDOW

Submitting a request grants reviewers **read** access to the proposer's own source course — opening on submit, closing the moment the request settles. A *consented exception* to "a personal course is private even from admins", not a bypass. Only the proposer's own course can be a source: refused at INSERT, re-checked in the helper, re-checked again inside the apply RPC.

The Requests view serves three audiences **by capability, not by role name**: deciders (admins *and* owners of the targeted standard course) get the full queue with live diff counts; managers get a read-only queue; proposers get their own requests with the reviewer's note verbatim, sorted so `changes_requested` comes first.

Managers get the request row — summary, status, outcome — but **never the fork content**.

10 Who talks to whom

IN PLAIN TERMS

Every connection in the system, in one scannable list — 29 of them. The colour on the left of each source tells you which world that connection lives in: the Windows program only, both apps, the caretaker console, inside the back office, or scheduled machinery running without anyone present. Underneath is the list that matters just as much: **the connections that deliberately do not exist**. In a security conversation, that second list is usually the answer.

DESKTOP-ONLY BOTH HOSTS OPERATOR SERVER-SIDE OUT OF BAND

#	SOURCE		DESTINATION	PROTOCOL	PAYLOAD
01	Electron main	→	Renderer	loopback HTTP	Starts the Express server on an ephemeral port and points the BrowserWindow at it — the renderer's own origin IS its API origin. No port is ever passed over IPC.
02	Renderer	→	Local Express	HTTP, same origin	O.T.T.E.R. local content, RABBIT project bundles, files and managed files, thumbnails, rate cards, team members, task templates, pet and settings, plus URL fetch / raw fetch / PDF extract.
03	Renderer	→	Electron main	IPC	Window controls, zoom, OS file and folder pickers, file copy with progress, thumbnail generation, Google Drive credentials, local-data archive, update check/download/install, and the safeStorage session slot.
04	Local Express	→	Disk	fs	{userData}/otter-data/ and {userData}/rabbit-data/, plus the user-visible project folder when configured.
05	Renderer	→	Supabase Auth	HTTPS	signInWithPassword, mfa.challenge / mfa.verify, refreshSession, resetPasswordForEmail, updateUser. The access-token hook runs inside Auth at issuance.
06	Renderer	→	PostgREST	HTTPS + bearer	Direct table reads and writes — the main data path in cloud mode. Safe because RLS is the boundary.
07	Renderer	→	Postgres RPCs	HTTPS + bearer	Soft delete and restore, the O.T.T.E.R. fork/apply/index family, the member directory, the trash index. These exist because a plain statement cannot express the check safely.
08	Renderer	→	Supabase Storage	HTTPS	Avatar upload to user-avatars; project file upload and download in rabbit-files.
09	Renderer	→	Supabase Realtime	websocket	Joins rabbit:project:{id} and rabbit:workspace:{id}, authorised once at join; receives full old/new row payloads and presence.
10	Postgres	→	Realtime	in-database	AFTER triggers call realtime.broadcast_changes on 10 project-scoped and 5 workspace-scoped tables.
11	Renderer	→	resolve-login · issue-session	HTTPS	Unauthenticated and bearer respectively, during sign-in and workspace switching.
12	Renderer	→	provision-workspace	HTTPS	Unauthenticated, from the new-company wizard.

13	Renderer	→	invite-member · admin-create-user · admin-reset-password · admin-set-active · admin-user-security	HTTPS + bearer	From the Admin Terminal and Team Members, each with the caller’s own token.
14	Renderer	→	storage-gc	HTTPS + bearer	From Diagnostics, on an admin’s click.
15	Renderer	→	ai-proxy	HTTPS + bearer	Every AI feature in all three tools, the companion and the agent — the single path to Anthropic.
16	Operator console	→	operator-workspaces · operator-ai-keys · PostgREST	HTTPS + bearer	And, uniquely, direct PostgREST for the platform_audit read — gated by RLS alone.
17	Every Edge Function	→	Postgres	service_role	Bypassing RLS — which is why each one re-checks a live row itself.
18	Edge Functions	→	GoTrue admin API	HTTPS	Create user, update user, invite by email, list MFA factors, ban, sign out.
19	ai-proxy	→	api.anthropic.com	HTTPS, SSE	Per-workspace key decrypted from workspace_ai_keys, or the platform key. Always streaming.
20	operator-ai-keys	→	api.anthropic.com	HTTPS	A one-token validation call when a company key is set.
21	operator-workspaces	→	Supabase Storage	service_role	Sweeps rabbit-files during teardown.
22	Supabase Auth	→	Resend	SMTP	Invite, recovery and email-change mail.
23	pg_cron	→	Postgres	in-database	The five nightly purge functions.
24	GitHub Actions	→	wilson-dev	HTTPS	The issue-session smoke probe and the Playwright lanes.
25	GitHub Actions	→	Backblaze B2	S3 API	Nightly pg_dump upload — from the default branch only.
26	Vercel	→	GitHub	webhook	Builds both surfaces on every push to the production branch, with staging’s URL and anon key as build-time variables.
27	electron-updater	→	Backblaze B2	HTTPS	Checks the generic feed at login; downloads the NSIS installer on request.
28	Renderer + Electron main	→	Sentry	HTTPS	Independently, per environment.
29	R.A.B.B.I.T. (desktop)	→	Google Drive API	HTTPS, read-only	Using tokens held in rabbit-data/.

ARROWS THAT DELIBERATELY DO NOT EXIST

- ✗ No client → Anthropic. Ever, on either host.
- ✗ No desktop app → operator console. It is not in the build.
- ✗ No cross-tenant table read except `operator_workspace_summary()`.
- ✗ No `postgres_changes` subscriptions.
- ✗ No customer content → Backblaze B2.
- ✗ No O.T.T.E.R. or Notes content on any realtime topic.
- ✗ No O.T.T.E.R. or Notes content in the company takeout.

EXACTLY ONE CROSS-TOOL EVENT EXISTS

`wilson:open-otter-course`

Dispatched by the Admin Terminal's change-request section after probing that the course is readable, and listened to by both `App.jsx` (which navigates the shell) and `Otter.jsx` (which re-probes, loads and selects the course). **All other cross-component signalling uses props, counters or context.**

The Otter listener is deliberately *not* gated on `currentPage`, unlike the keyboard shortcut — it only ever fires from an explicit admin click, and handling it while hidden is the entire point.

11 The invariants index

the things a reader must not casually change

> IN PLAIN TERMS
>
> One page listing everything in WILSON that looks like an ordinary implementation choice but is not. Each one has a reason next to it. If you are about to change something on this page, the reason is what you need to argue with — **not the code**.

FROZEN §4.3

The JWT claim shape

Every RLS policy on a tenant-scoped table starts from `workspace_id` via `current_workspace_id()`. Changing the four key names means rewriting RLS on every domain table *and* re-issuing every live token.

FROZEN §13.3 · §13.4

roleMatrix.js ⇄ SQL helpers, in lockstep

The client matrices mirror the SQL helpers exactly. A unit test fails on any mismatch *in either direction*, and the migration's `COMMENT ON FUNCTION` points back at the JavaScript file by name.

FROZEN §13.1

D.O.G.'s three canonical export filenames

Downstream tooling consumes them. The format is locked by default rather than immutable — editable behind `formatTabLocked` / `promptsTabLocked`, both defaulting true.

CLOSED §5.3

operatorGuard

No verified MFA factor refuses; a failed factor lookup refuses. An auth check that cannot verify has no excuse — **and this surface can delete a tenant**.

CLOSED §4.6

adminGuard MFA step-up

A failed or errored `listFactors` returns `503 mfa_check_failed`. The `error` channel is checked as well as thrown exceptions, because supabase-js reports most failures there.

CLOSED §3.1

wilsonSession safeStorage bridge

When the OS keychain is unavailable, `save` returns `{ok:false, reason:'no_keychain'}` and `load` returns null. **None of the three ever falls back to plaintext.**

CLOSED §4.7

fn_try_uuid on storage paths

A garbage path segment fails closed, where `can_write_project(NULL)` alone would fail *open*.

OPEN §4.6

The Edge rate limiter

Fails open, loudly. *A limiter is an abuse control, not an authorization control, and authorization has already run above it* — a database hiccup must not take every tenant's AI features offline. Compare `operatorGuard`: both choices were made in the same session, on purpose.

OPEN §6

ai-proxy key fallback

A failure decrypting a company key falls through to the platform key. *A tenant key is a billing preference, not an authorization boundary.*

ORDER §5.4

Workspace teardown, 8 steps

Collect blob paths **before** the CASCADE — afterwards nothing can tell which blobs were the tenant's. Drop queue rows **after** — the CASCADE re-enqueues them.

ORDER §12.2

Relink's three rungs

Each rung only sees what the rungs above left unclaimed, and `strong` runs as a full pass before `name` so a size-mismatched row cannot steal a candidate.

ORDER §13.2

otter_cr_apply()

Archive the target first, *then* apply additively, *then* flip status. Approving **applies**; a bare status flip raises. Additive only: update by slug, insert when absent, never delete.

FROZEN §4.9

Migration replay pairs

Re-running `0022` by hand requires `0025` *and* `0026`; re-running `0002` requires `0029`. Every 0022 post-condition still passes in the half-reverted state.

CLOSED §5.2

The session key split

`__WILSON_SURFACE__` is a build-time constant, not a runtime branch. Any code that reads a session key without going through `sessionStorage.js` reintroduces the leak. **There is no second mechanism to catch it.**

FROZEN §5.3

No grant-or-revoke-operator endpoint

Writes to `platform_operators` exist in three places repo-wide: one SQL runbook and two pgTAP fixtures. The highest privilege in the system cannot be escalated from a web session *even by someone already holding it*.

CLOSED §4.4

Capture triggers never abort their write

`EXCEPTION WHEN OTHERS → RAISE WARNING`. An audit failure must never abort the write it audits — including a workspace CASCADE.

FROZEN §4.2

verify_jwt = false on all twelve functions

The gateway only supports HS256; these tokens are ES256. Turning it back on breaks, it does not harden.

ORDER §7.2

Scheduled workflows live on the default branch

GitHub fires `schedule` workflows **exclusively** from the default branch. Any future scheduled workflow must live there or it is decoration.

12 Security posture, and what is known to be missing

handbook §15 · §16 · §17

IN PLAIN TERMS

An honest scorecard. The technical protections are strong and independently tested; the paperwork around them — written policies, vendor checks, incident response — has barely started, and that is where a studio audit spends most of its time. The items in red are known and tracked, not discovered by whoever is reading this. **Nothing here is hidden from a client conversation**; being able to name your own gaps precisely is usually worth more than a clean-looking summary.

TPN RE-AUDIT · 2026-07-30 · MPA CSBP 5.3/5.3.1

Shield tier eligibility: None — but for the first time the blockers are *countable* rather than structural.

3	43	31	7	6
CRITICAL	HIGH	MEDIUM	LOW	RESOLVED

All three of the April baseline's immediate-action items are closed. Authentication moved from a single shared password to Supabase Auth with TOTP, a four-tier role model and RLS-enforced RBAC pinned by pgTAP in CI. Content lifecycle went from nothing to an append-only per-file event stream with deletion certificates that outlive the workspace they belonged to. The Anthropic key is out of client storage entirely.

The honest framing for a studio conversation: the *technical* controls have improved enormously and are close to defensible; the *organizational* ones — policies, vendor risk, incident response, supply chain — are essentially untouched, and those are where a Silver or Gold assessment spends most of its time.

THE TWO OPEN CRITICALS — BOTH OWED

1 · A live workspace-admin credential was published in this public repository.

Session 15 removed every occurrence from the working tree and rewrote the instructions that told operators to rotate the password *back* to that literal — which is why it stayed valid for eleven sessions. **The value is in git history permanently, so rotation is the only remedy.**

2 · Privilege changes are unaudited. TPN-LOG-005

Role promotions, rate-card grants and deactivations go from the browser straight into `workspace_members`, and nothing captures them: edit history deliberately excludes the table, the only triggers on it are guards, and no `app_events` line is written. **"Who granted this person admin, and when" is unanswerable for anything that already happened** — this is the audit trail for the product's own security boundary.

NAMED STRONG POINTS

Table-layer tenancy — Cloud and Content are the strongest domains · four append-only RLS-enforced audit streams · per-tenant AI keys as correctly-implemented AES-256-GCM with a 32-byte key check and a per-record IV · the operator tier's SQL-only grant.

NAMED WEAK POINTS

Logging has no aggregation and no alerting; sign-in and operator-console access are unlogged · `app_events` and `edit_history` purge at 90 days against a 1-year requirement · the local server's bare CORS and unvalidated URL fetchers · the public `user-avatars` bucket's unconditional SELECT · Google OAuth material stored in plaintext on disk despite safeStorage being correctly wired for the session · Third-Party and Documentation, unmoved since April.

KNOWN LIMITS — TRACKED, NOT DISCOVERED

docs/MASTER_PLAN.md §6 · TPN_AUDIT/FINDINGS.md · docs/OWED_AUDREY.md

SECURITY-ADJACENT

`managed-files` PATCH merges the body with no field stripping, and its hard-delete branch joins paths onto the project root with **no containment check** — unlike the sibling `files` routes, hardened for exactly this.

`custom_access_token_hook` is executable by `authenticated` and `anon`: 0011's blanket grant silently re-grants what 0001 and 0003 revoke, and it takes its target user id from the caller-supplied argument.

`invite-member` performs no MFA step-up and can create an `admin`.

Operator sign-in, sign-out and guard refusals write no audit row anywhere, and `platform_audit`'s action CHECK has no value that would let them.

CORRECTNESS

R.A.B.B.I.T. **milestones are dropped on project load** — the routes exist, but `loadProject` omits the key.

"Add files" inside an entity popup throws in cloud or Drive mode; `createManagedFile` exists only on the local adapter.

`googleDriveAdapter` wires rate-card reads to its read-only throw stub — a permanent error banner in Drive mode.

A username that collides across two workspaces makes sign-in unreachable: the resolver treats two matches as a miss, and the login screen has no slug field.

O.T.T.E.R.'s Space-to-search listener is not gated on `currentPage`.

`logAdminEvent` hardcodes `severity: 'info'`, so `WIL-3004` "failed" is indistinguishable from success in the log stream.

PRODUCT GAPS THAT READ AS BUGS

Quiz scores and Validator findings are **never persisted** — the columns, routes and adapter operations all exist server-side, but the client never calls them.

Settings → Tools "Storage Location" is editable and has no effect; `getDataDir()` hardcodes the userData path.

R.A.B.B.I.T.'s agent integration is prompt-selection only — neither its `registerTool` call nor its tool factory has a call site.

D.O.G. cloud attachments are refused at the write layer, with the re-homing plan and its seven traps catalogued for S17.

ACCEPTED FOR v1.0.0, WITH REASONS

GC orphan-scan starvation — the run-wide budget collects in name order. No tenant is near 5000 referenced objects. The sharper edge is that certified disposal only runs when a human clicks, with no queue-depth signal anywhere.

Synchronous fs on the Electron main process during relink: an unreachable share can freeze the app. An availability defect on a local, user-initiated action.

Teardown cannot see blobs no row points at. The row-derived sweep covers every blob the product itself created.

No single-instance lock in Electron; two copies can run against one `userData` directory.

Web multi-tab last-writer-wins on the three localStorage stores. Accepted for a one-window product.

EVENT AND ERROR CODES

src/cloud/errorCodes.js — WIL-41xx and the 'admin' type are server-reserved so clients cannot forge audit lines

WIL-1xxx Authentication — declared, largely unwired. `app_events`	**WIL-3003 / 3004** Storage cleanup completed / failed. `app_events`	**WIL-41xx** Admin actions, server-written by `logAdminEvent`. `app_events`	**WIL-5001 / 5002** Update check / download failure. `app_events + Sentry`	**WIL-6001 / 6002** AI request completed / failed — model, tokens, `key_source`. `app_events`
WIL-7005 `workspace.teardown` certificate — **critical**. `platform_audit`	**WIL-7006** `blob.purged` batch certificate. `platform_audit`	**WIL-7007** Teardown failure. `platform_audit`	**WIL-7008** Teardown refused foreign paths. `platform_audit`	**WIL-7010 / 7011** Company AI key set / cleared — **hint only, never the key**. `platform_audit`

This document is the visual counterpart to **SYSTEMS_HANDBOOK.md**, not a replacement for it. Every name here is the real one. Where this document and the handbook disagree, the handbook is right; where the handbook and the code disagree, **the code is right — and the handbook needs a fix**.

WILSON Systems Atlas · v1.0.0
Petal Studios · 2026-07-30
from the frozen post-Session-15 tree
commit 09b4405 · feat/multi-user-v1